27｜AI 写代码，保证质量不靠运气，靠体系

Robert · Claude Code 企业级全链路开发实战



你好，我是 Robert。

Anthropic 公开说过：Claude Code 自身的代码，全部由 AI 编写，工程师只负责 review。在我看来，这句话有一个没说出来的前提：**他们敢这么做，不是因为 AI 不会出错，而是因为有一套完整的质量保障体系**。

在说这套体系之前，我想先说清楚一个判断。

现在很多新闻告诉我们：写好 SDD，直接扔给 AI，它跑一个晚上就把整个项目搞定了。我的个人体感是：做不到，至少现在做不到。

不是 AI 能力不够，而是这句话偷换了一个概念：**保证代码质量的不是 AI，是体系。AI 只是体系的执行者**。

这两件事的区别很大。

“AI 帮你保证质量”的意思是：你把代码交给 AI，它会自动发现所有问题，自动写出正确的测试，自动保证系统稳定。这是不现实的。AI 写代码时专注于功能实现，它不会主动质疑自己的安全模型，不会想到“这个接口的权限校验是不是有漏洞”，不会考虑并发场景下的数据一致性。

“体系保证质量，AI 协助执行”的意思是：你先建立质量保障体系，**识别核心链路**、**划定测试边界**、**制定 review 策略**，然后让 AI 在这个体系里高效执行。体系是你建的，AI 是执行者。

一个项目的质量由四个层次组成。

1. **核心链路识别**：哪些是系统的核心路径，哪些是风险集中的地方。这个判断必须由人来做，AI 辅助分析。做错了，后面所有测试都是在错误的地方使劲。
2. **单元测试**：覆盖纯逻辑，不依赖外部系统。这部分独立性强、模式固定，AI 能规划得很好，可以大比例交给 AI 执行。但要注意：AI 偏逻辑完备，测试用例容易过多，你要把握边界，砍掉不必要的。
3. **集成测试**：功能维度的端到端验证，从 Controller 到数据库的完整链路。mock 策略要你来定，执行让 AI 来做，一个场景跑通再写下一个。
4. **混沌测试**：故障注入验证韧性，是质量体系的天花板。单测和集成测试验证“正常情况下对不对”，混沌测试验证“出问题时垮不垮”，节点宕机、网络分区、磁盘打满、慢消费者积压。这一层在项目维护阶段才有意义，但要从架构阶段就知道它在哪里。

所以，正确的工作方式不是告诉 AI：“给我写测试用例，保证代码不出问题”，然后等它跑完。而是你来拆解项目，划定边界，AI 在你划定的边界里执行。

这一讲围绕就这个思路展开。本节课分两部分：**方法论说清楚怎么想**，**实操给你能直接用的提示词和流程**。

方法论

判断：哪些代码该给 AI 写，哪些自己把关

质量保障的起点不是测试，是判断。我有两个观点：

适合给 AI 写的：有规范、有模式、有标准答案的代码。

CRUD 接口、适配器、DTO 转换、工具类、配置类、测试代码，这类代码逻辑清晰，有套路可循。AI 写出来和有经验的程序员写的差不多，而且更快。Hify 整门课 80% 以上的代码属于这类，全部放手给 AI。

不适合给 AI 写的：需要深度判断、没有标准答案的代码。

架构设计、核心数据模型的取舍、安全边界的划定、性能瓶颈处的核心逻辑，这类代码 AI 能写出来，也能跑，**但方向可能是错的**，而且你不一定能从代码本身看出来。错的方向比写错的代码危险得多，因为它表面上没问题。

Hify 里的分工就是这样：Provider 的 CRUD、适配层的格式转换、前端的管理页面，全部放手给 AI。04 讲的数据模型设计、13 讲的领域理解、整套架构的模块拆分，这些是自己做的，AI 只是辅助分析。

做出上面的这个判断不难。有经验的工程师看一眼系统，基本知道哪里容易出问题。核心链路、核心功能、核心模块等这些是要把关的地方，也是 review 和测试的重心所在。判断做对了，精力分配就对了。

Review：按核心链路看，双模型互补

代码量大，逐行 review 不现实，**核心链路 review 才是正确姿势。**

这里有一个前提：结构组织要合理，review 才有抓手。模块清晰、分层合理、CLAUDE.md 写清楚规范，结构好了，你知道核心链路在哪几个文件里，review 变成有目的的阅读，而不是大海捞针。所以到这里你会发现，我们在前面的课程沉淀了很多内容到 CLAUDE.md 中的作用来了。

Hify 的核心链路是对话链路，`ChatServiceImpl` 里从用户发消息到 LLM 返回的完整流程。所以需要重点关注这条链路。

我在平时编码，有个习惯：就是双模型互补。

就是 Claude Code 写完核心模块，把代码喂给 ChatGPT review。两个模型训练方式不同，盲点不同，互补效果很好。

所以我其实不推荐你买 Claude Code，大家去买 Cursor 也可以。因为 Cursor 集成了 ChatGPT、Claude Code、Kimi 等等多个模型，很适合多模型 review。

双模型 review 的提示词可以这么给：

```
你是一位有十年经验的 Java 后端工程师，正在做代码 review。
以下是 Hify 对话链路的核心代码，请从这几个维度给出意见：

1. 安全性：权限校验是否完整，有没有越权访问的可能
2. 静默失败：有没有 catch 块吞掉了异常，调用方感知不到失败
3. 边界条件：null 处理、空列表、并发场景有没有遗漏
4. 性能隐患：N+1 查询、不必要的阻塞调用、大对象

不需要夸代码写得好。直接说问题，每个问题给出具体位置和修复建议。

代码如下：
[粘贴核心链路代码]
```

加“不需要夸代码写得好”是因为模型有讨好倾向，不说这句会先说一堆好话再说问题。举个例子，我把 `ChatServiceImpl` 喂给 ChatGPT，它发现了 12 个问题，4 个高危：

1. 越权访问他人会话：`ChatController` 所有带 sessionId 的接口，只验证 session 存在，不验证 session 属于谁。用户 A 知道用户 B 的 sessionId（连续自增 ID，极易枚举），可以读取 B 的全部对话历史、向 B 的会话发消息、删除 B 的会话。
2. 对话历史取的是最旧的消息：`ChatMessageMapper` 的 SQL 是 `ORDER BY created_at ASC LIMIT #{limit}`。对话有 100 条时，取的是最早的 20 条，不是最近的 20 条。长对话用户发新消息，LLM 看到的是几十轮之前的历史。
3. MCP 工具调用失败静默返回 mock 数据：`executeToolCall` 里 catch 所有异常后返回硬编码的 mock 数据。MCP Server 超时、认证失败、工具不存在，全都返回一样的假数据，调用方完全感知不到失败。

4. Redis read-modify-write 无锁：`updateContext` 是 get → add → set 的操作序列，没有任何锁。两个并发请求同时读到旧的 context，各自追加后写回，后写的覆盖前写的，一条对话记录永久丢失。

从效果来看，还是挺好的，确实找出了实际问题。为什么呢？**Claude Code 写代码时专注于功能实现，它不会主动质疑自己的安全模型**。ChatGPT 扮演 reviewer 时，视角完全不同，它在找问题，不是在完成功能。从实践来看，这个操作的实际效果挺好的。

单测边界：先问该不该测，再问怎么测

让 AI 决定测什么是错的起点。AI 偏逻辑完备，倾向覆盖所有路径，包括 mock 掉外部依赖之后意义不大的场景。

> **值得写单测的**：输入固定输出就固定、不依赖任何外部系统。Hify 里是两类：纯函数逻辑（`buildMessages()` 上下文拼装、`ConditionNodeExecutor.execute()` 条件判断、`ExecutionContext.resolve()` 模板替换）和边界清晰的工具类（`NodeConfigParser.parse()`、`Result.ok()` / `Result.fail()`）。
>
> **不值得写单测的**：核心是外部调用的代码（`OpenAiAdapter`，用 MockWebServer 做 HTTP 级集成测试更有价值）；逻辑在 SQL 里的代码（`selectRecentBySessionId` 的排序 bug 只有 H2 集成测试能发现）；深度依赖 Spring 容器的代码（`ChatServiceImpl` 有 12 个外部依赖，mock 完之后剩下的只是调用顺序）。

判断三问：

```
1. 有没有 IO（DB/HTTP/Redis/文件）？
   有 → 考虑集成测试

2. 去掉外部依赖，剩下的逻辑有复杂度吗？
   没有 → 不值得测
   有 → 可以单测

3. 改动频率高？出错影响大？
   是 → 值得做
   否 → 优先级放低
```

Hify 实操

接下来我们根据这些方法论，对 Hify 进行测试，主要分为五步。

1. 识别核心链路，写入 CLAUDE.md

所有测试的起点不是写代码，是知道测什么。先让 AI 读完整个项目，输出核心链路清单和风险地图，你来确认，写进 CLAUDE.md。后面的单测和集成测试都基于这份地图来做。

提示词：

```
帮我分析这个系统，输出三个清单：

1. 核心链路清单（3-5条）
   每条链路：名称、涉及的模块和类、为什么是核心链路

2. 风险集中区域
   哪些模块/方法最容易出问题、出了问题影响最大
   每个风险点说明：风险类型（安全/并发/性能/数据一致性）、可能的失败场景

3. 测试重心建议
   基于前两条，测试精力应该往哪放
   哪些地方必须有测试覆盖，哪些可以先跳过

输出格式：结构化的 CLAUDE.md 片段，我直接复制进去。
```

Claude Code 读完代码后，输出的核心链路清单和风险地图相当准确。以 Hify 为例，它识别出了 5 条核心链路：

> **流式对话链路（最高优先级）**：`ChatServiceImpl.doStreamChat()` 串联了 12 个外部依赖，RAG 检索、Function Calling 两轮 LLM、Redis/MySQL 双写全部在一个方法里。
>
> **Function Calling 两轮链路**：tool_calls 分支是 Hify 区别于普通聊天的核心能力，失败时静默 fallback mock 数据，调用方无法感知。
>
> **消息上下文管理链路**：`loadContext()` + `updateContext()` + SQL，多轮对话的正确性依赖它，有已知 bug（SQL 取最旧而非最新消息）。
>
> **工作流执行链路**：`WorkflowEngine.pickNext()` 有 fallback-to-first-edge 兜底逻辑会掩盖配置错误，CONDITION 节点走错分支时静默终止无报错。
>
> **MCP 工具接入链路**：每次对话同步串行探测所有绑定的 MCP Server（5s 超时 × Server 数量），3 个 Server = 15s 阻塞。

风险地图识别出了 7 个风险点，其中 2 个高危：

> `ChatServiceImpl.doStreamChat()`，工作流路径抛 RuntimeException 时 user 消息落库但 assistant 消息缺失，对话历史出现孤儿消息；MCP 工具调用任何异常均 fallback mock 数据，LLM 基于假数据生成回答，用户无感知。
>
> `ChatMessageMapper.selectRecentBySessionId()`，SQL 是 `ORDER BY created_at ASC LIMIT #{limit}`，长对话时取的是最早的消息而非最近的，LLM 拿到过时上下文，对话质量急剧下降。短对话时完全无症状，只在用户深度使用时暴露。

这时候你就要做的一件事，就是：**拿到结果，对照自己对系统的理解，检查 AI 有没有遗漏或判断错的地方。AI 的分析基于代码结构，你的判断来自业务理解，两者对比才能确认这份地图是准的。**

确认后写进 CLAUDE.md。这份地图会影响后面每一步，单测规范基于它生成，集成测试的 P0 优先级基于它划定，混沌测试的场景设计也基于它。

2. 生成单测规范，写入 CLAUDE.md

有了核心链路地图，单测规范就有了依据。让 AI 基于 Step 0 的结果生成规范，针对 Hify 这个具体项目，而不是泛泛的通用规范。

提示词：

```
基于 CLAUDE.md 中的核心链路和风险地图，帮我生成 Hify 项目的单元测试规范。

规范需要覆盖：
1. 哪些代码必须写单测（结合核心链路判断）
2. 哪些代码不写单测、用集成测试替代（结合 Hify 的外部依赖特点）
3. 测试命名规范：should_[期望结果]_when_[输入条件]
4. 测试结构：Given-When-Then
5. mock 使用规范：什么时候 mock，什么时候不 mock
6. 断言规范：用 AssertJ，断言要有意义
7. 禁止事项：哪些写法不允许出现

输出格式：直接输出 CLAUDE.md 片段，我复制进去就能用。
```

Claude Code 先检查了项目的测试依赖，发现根 pom 里没有任何测试框架：零测试基础设施。所以它的规范里第一节就是依赖配置。

这个细节很重要：不是泛泛说“用 JUnit 5”，而是**先看你现在有什么，再告诉你缺什么**。

规范里有几个值得关注的判断。关于“不写单测”的范围，它给出了具体理由而不是笼统结论：`ChatServiceImpl`整体不写，因为有 12 个外部依赖，mock 掉之后测的是假数据流转；`OpenAiAdapter` 不写单测，因为核心是 HTTP 构造和 SSE 流解析，与网络协议深度耦合，用 MockWebServer 做 HTTP 级集成测试更有价值。

关于 mock 使用规范，它强调一个原则：**只 mock 跨越本类边界的依赖，不 mock 标准库、不 mock 被测类自身**。举了一个反例，`@Mock ObjectMapper objectMapper`，直接 new 即可，不需要 mock。

禁止事项里有一条特别有用：**禁止在测试里写业务逻辑，不要重复计算期望值，直接写字符串字面量**。这是 AI 写测试时最常见的坏味道，它会把被测的计算逻辑在测试里再实现一遍，测试通过只能说明两段逻辑一样错。

生成后加进 CLAUDE.md 的“测试规范”节，此后 Claude Code 写任何测试代码都会自动遵守。

3. 分析哪些模块需要单测，给出清单

规范写好了，让 AI 读完所有相关代码，对照规范输出清单。注意提示词里要让它先读代码再分析，不是凭感觉判断。

提示词：

```
基于 CLAUDE.md 中的单测规范和核心链路地图，分析 Hify 以下模块，
给出"值得写单测/不值得写单测"的清单。

对每个类/方法，输出：
1. 是否值得写单测（是/否）
2. 理由（一句话）
3. 如果值得测，列出 2-3 个最重要的测试场景

用表格输出，按模块分组。不要写测试代码，先给清单。
```

Claude Code 读完代码后给出了 27 个判断目标，结论是 17 个值得写、10 个不值得写。你要做的一件事，拿到清单做最终取舍。**清单告诉你重心在哪，但优先级你来定**。

Claude Code 建议的写测试顺序是：先修复已知 bug（SQL 问题）同步写回归测试锁住正确行为，再给纯函数写单测，最后做集成测试链路。这个顺序是合理的，先堵已知漏洞，再建体系。

4. 单测：模型管理模块 & 固化为 SKILL

有了清单，开始做。先手动走一遍 Provider 的单测，跑通之后把流程固化成 Skill。这里完整展示和 Claude Code 的交互过程，包括它主动纠正我的两个地方，这才是真实的节奏。

先规划，不直接写代码

```
深度分析 ProviderService 的 createProvider 方法。

告诉我：
1. 这个方法有哪些执行路径（正常路径 + 异常路径）
2. 每条路径的关键变量是什么
3. 哪些边界条件最容易出错

基于分析，给我一份测试计划：测哪些场景、每个场景验证什么断言。
先给计划，不要写代码。Claude Code
```

Claude Code 读完代码，画出了完整的执行路径树：

```
create(request)
    │
    ├─ checkNameUnique(name, null)
    │    ├─ 路径 A：DB 查到同名 → 抛 BizException(PROVIDER_NAME_DUPLICATE) ✗
    │    └─ 路径 B：name 不重复 → 继续
    │
    ├─ new Provider()，逐字段赋值
    │    └─ description == null → 写入 ""（空字符串兜底）
    │
    ├─ provider.setEnabled(1)（硬编码，调用方无法控制）
    │
    └─ providerMapper.insert(provider)
         ├─ 路径 C：insert 成功 → id/时间戳由 MyBatis-Plus 回填 ✓
         └─ 路径 D：insert 失败 → RuntimeException 穿透到 GlobalExceptionHandler ✗
```

同时发现了三个边界条件值得关注：`checkNameUnique` 里没有显式过滤软删除记录（依赖 MyBatis-Plus `@TableLogic` 自动追加）；`description` 传全空格 `" "` 时原样入库没有 trim；`@CacheEvict(allEntries=true)` 的范围过宽，清空了整个 provider-cache 而不只是 list 缓存。

测试计划给出了 11 个场景，分 P0/P1/P2 三档，还有两条标注了“只能用集成测试验证”。

reveiw 测试计划，做筛选

拿到计划，我砍掉了 P2 的场景，留下核心的四条，然后发给 Claude Code 让它写代码：

```
测试计划确认如下：
- 正常创建：provider name 唯一，验证返回 id 且数据库有记录
- 重复名称：抛出 BizException，错误码 PROVIDER_NAME_DUPLICATE
- 必填字段为空：name 为 null，抛出 ConstraintViolationException
- apiKey 格式校验：不符合格式，抛出 BizException

按这个计划写单元测试。

技术约束：
- ProviderService 依赖 ProviderMapper，用 @MockBean mock 掉
- 用 AssertJ 做断言，不用 assertTrue
- 每个场景一个独立测试方法，方法名遵循 CLAUDE.md 规范
- 不要写集成测试，只写单元测试
```

这是这一步最值得关注的地方。Claude Code 没有照着计划直接生成代码，而是先指出了三个问题：

> **“验证返回 id 且数据库有记录”在单元测试里做不到后半句。**
> `providerMapper.insert()` 被 mock 掉后，数据不会真正写库，“数据库有记录”只能在集成测试里验证。单元测试能验证的是：`insert()` 被调用了一次，且传入的 provider 对象字段正确。
>
> **“apiKey 格式校验”在代码里根本不存在。**`authConfig` 是 `Map<String, Object>`，只有 `@NotNull` 约束，没有任何格式校验逻辑。这条测试没有被测对象，应该改成“authConfig 为 null 时 Bean Validation 拦截”。
>
> **`@MockBean` 和“只写单元测试”冲突。**`@MockBean` 需要启动 Spring Context，属于集成测试范畴。真正的单元测试应该用 `@Mock + @InjectMocks + @ExtendWith(MockitoExtension.class)`，毫秒级启动。

值得一提的是：上面的三个点有点太细节了，但是我希望你看到的是 Claude Code 输出的是什么样子的提示，而不是写了什么。你应该学的是它的思维，而不是具体的输出。

单测流程固化为 SKILL

最后流程跑通，固化为 SKILL：

```
我刚才做了 ProviderService 的单元测试，完整流程是：
1. 读代码，分析执行路径和边界条件
2. 输出测试计划（不写代码）
3. 我确认计划，CC 主动指出计划里的技术问题，调整后再执行
4. Service 业务逻辑和 DTO 约束规则分两个测试文件
5. 跑测试，失败了分析是测试写错还是实现有 bug

请把这个流程固化成一个 SKILL.md 文件。
让我以后对任何 Service 方法用 /单测 命令触发，CC 自动按这个流程走。

SKILL.md 要包含：
- 触发方式
- 读代码的步骤（先读被测类，再读依赖的 DTO 和 ErrorCode）
- 测试计划的输出格式（执行路径树 + 边界条件 + 分优先级的场景表）
- 写代码前的技术确认清单（mock 方式、断言库、是否有 Bean Validation 相关场景）
- 测试文件拆分原则（Service 逻辑和 DTO 约束分开）
- 跑测试和处理失败的步骤
```

固化后放在 `.claude/skills/unit-test.md`。下次对任何方法直接 `/单测 AgentService.createAgent`，Claude Code 自动走完整流程，包括主动检查计划里的技术问题。

5. 集成测试 & 固化为 SKILL

集成测试是 Hify 的主力。Spring Boot Test 完整启动，mock 掉外部 API，打真实 HTTP 请求，走完从 Controller 到数据库的完整链路。还是先规划集成测试清单：

```
基于 CLAUDE.md 中的核心链路和风险地图，帮我规划 Hify 的集成测试清单。

测试范围：Spring Boot Test + MockMvc，mock 掉 LLM API 和 MCP Server，
```

```
测试 Hify 自身从 Controller 到数据库的完整链路。

按优先级分三档：
P0，核心链路，必须覆盖
P1，主要功能，应该覆盖
P2，边缘场景，有余力再做

每条给出：测什么、验证什么、为什么这个优先级。
不要写代码，先给清单。
```

Claude Code 先检查了项目的配置文件，发现已经有 `application-mock.yml`，H2 内存库 + MockProviderAdapter 替换真实 LLM 调用，不需要 Redis。技术基础已经就位，不需要额外搭建。

Claude Code 给出了 13 条清单，P0 五条值得细说：

完整对话链路：普通问答，验证 SSE 流包含 `type=delta` 事件且 content 非空，最后一个事件是 `type=done`，MySQL 里有 user 和 assistant 两条记录，且 assistant 消息内容与 delta 事件拼起来一致。

Function Calling 两轮链路：MockProviderAdapter 第一轮返回 `finish_reason=tool_calls`，mock McpService 返回工具结果，第二轮返回最终回答。验证 SSE 流正常结束，MySQL 只落一条 assistant 消息（不是两条），`McpService.callTool()` 被调用一次。重点是验证工具调用失败时不影响 SSE 流正常结束，这正是 review 环节发现的高危问题：工具失败静默 fallback mock 数据。

对话上下文多轮正确性：同一 session 发 3 条消息，验证第 3 条调用时传给 LLM 的 messages 数组包含前两轮历史，且顺序正确。这条直接对应 Step 0 识别的已知 bug，SQL 取的是最旧消息而非最新。Claude Code 建议先写这条测试，再去修 bug，修完后测试从红变绿。这是测试驱动修复，不是修完再补测试。

Provider CRUD 完整链路：创建 → 查详情 → 更新 → 删除，每步验证数据库状态，包括软删除后 list 接口不再返回该记录。

会话隔离：跨 session 越权访问，设计有点反直觉。当前代码没有 userId 隔离，测试会通过，但通过意味着越权访问可行。Claude Code 的建议是先记录现状，等加了权限校验之后这条测试会变红，变红才是对的。先用测试锁住现状，再推动修复。

测试数据策略：每个测试类用 `@Sql` 插入独立数据，用 `@Transactional` 回滚，不依赖全局种子数据。SQL 文件按场景分文件放在 `src/test/resources/sql/` 下，不要把所有测试数据堆进一个文件。

模型管理集成测试

清单确认后，从最简单的模块开始写。Provider CRUD 是前置依赖，后面所有对话链路测试都需要先有 Provider 数据，先把这块跑通。

发给 Claude Code 的指令如下：

```
按 P0 清单，先写1～4：Provider CRUD 完整链路的集成测试。

技术基础：用现有的 mock profile（H2 内存库 + application-mock.yml），
不需要新建 application-test.yml。

测试场景（6 个，先写完跑通再继续）：
1. POST /api/v1/providers 创建：合法请求，验证 body.code=200 且 body.data.id 有值
2. POST /api/v1/providers 重复名称：验证 body.code=2001（PROVIDER_NAME_DUPLICATE）
3. GET /api/v1/providers/{id} 查询存在的记录：验证返回完整字段
4. GET /api/v1/providers/{id} 查询不存在的记录：验证 body.code=2000
5. PUT /api/v1/providers/{id} 更新：验证数据库里的 name 确实变了
6. DELETE /api/v1/providers/{id} 删除：验证数据库里 deleted=1

约束：
- @SpringBootTest(webEnvironment = RANDOM_PORT) + MockMvc
- 每个测试方法用 @Sql 插入独立数据，用 @Transactional + @Rollback 回滚
- Hify 所有接口 HTTP 状态码都返回 200，错误码在 body.code 里
- 用 AssertJ 做断言
```

注意提示词里加了“Hify 所有接口 HTTP 状态码都返回 200”，如果不说，Claude Code 默认会断言 HTTP 400/404，而 Hify 的错误响应格式不是这样。这类项目约定要在指令里明确说清楚，不能靠 Claude Code 猜。

对话链路集成测试

Provider CRUD 跑通后，写对话链路。分三个场景递进，一个跑通再写下一个。接下来内容有点多有点细，我就不展开细节了，你主要看提示词。

场景一：纯对话

复制代码

```
写 01：完整对话链路，普通问答。

前置：复用 Provider CRUD 测试里的测试基类和 @Sql 数据准备方式。

测试步骤：
1. @Sql 插入 provider + model_config + agent + chat_session 数据
2. POST /api/v1/chat/sessions/{sessionId}/messages，body: {content: "你好"}
3. 收集 SSE 事件流（text/event-stream）

验证：
① SSE 流包含至少一个 type=delta 的事件，content 非空
② 最后一个事件是 type=done
③ 查询 chat_message 表，role=user 和 role=assistant 各有一条记录
④ assistant 消息的 content 与所有 delta 事件的 content 拼接结果一致

MockProviderAdapter 已经在 mock profile 里替换了真实 LLM，不需要额外 mock。
先写场景一，跑通再写场景二。
```

场景二：Function Calling 两轮链路

复制代码

```
写02：Function Calling 两轮链路。

在场景一基础上，Agent 绑定一个 mock MCP Server：
- @Sql 数据里加入 mcp_server + agent_tool 关联
- @MockBean McpService，配置 listToolsDetail() 返回一个工具 schema
- MockProviderAdapter 第一轮返回 finish_reason=tool_calls 的响应
- McpService.callTool() mock 返回工具执行结果
- MockProviderAdapter 第二轮收到 tool_result 后返回最终回答

验证：
① SSE 流正常结束，最后事件是 type=done
② MySQL 只有一条 assistant 消息（两轮 LLM 只落一次）
③ verify(mcpService).callTool(any(), eq("check_refund_eligibility"), any())

重点验证工具调用失败时：mcpService.callTool() 改为抛 RuntimeException，
```

```
16 验证 SSE 流仍然正常结束，不挂起，这是 review 环节发现的高危问题。
```

场景三：对话上下文多轮正确性（先写测试，再修 bug）

复制代码

```
1  写03：对话上下文多轮正确性。
2
3  这条测试对应一个已知 bug，先写测试，测试应该是红的，然后再修 bug。
4
5  测试步骤：
6  1. 同一 session 依次发 3 条消息（"第一条"、"第二条"、"第三条"）
7  2. 在第 3 条发送时，拦截 MockProviderAdapter 收到的 ChatRequest
8
9  验证 MockProviderAdapter 第 3 次收到的 messages 数组：
10 ① 包含"第一条"和"第二条"的历史消息（验证多轮上下文传递）
11 ② 历史消息按时间升序排列（先旧后新，符合 LLM 期望）
12 ③ 最后一条是"第三条"（当前用户输入）
13
14 这条测试现在应该失败，因为 selectRecentBySessionId 的 SQL 是
15 ORDER BY created_at ASC LIMIT，取的是最旧的消息而非最新的。
16 测试变红之后，再去修 SQL，修完测试变绿。
```

集成测试流程固化为 SKILL

复制代码

```
1  我刚才做了 Provider CRUD 和对话链路的集成测试，完整流程是：
2  1. 先规划测试清单（不写代码），基于 CLAUDE.md 的核心链路地图分 P0/P1/P2
3  2. CC 先读配置文件确认技术基础（H2/mock profile）再给清单
4  3. 从最简单的场景开始，跑通后再写下一个，不批量生成
5  4. 每个测试类用 @Sql 独立数据 + @Transactional 回滚，不共享数据
6  5. mock 策略：外部 API mock 掉，DB 用真实 H2
7  6. 对于已知 bug，先写失败的测试，再修 bug，让测试从红变绿
8
9  请把这个流程固化成 SKILL.md 文件。
10 以后对任何模块用 /集成测试 命令触发，CC 自动按这个流程走。
11
12 SKILL.md 要包含：
13 - 触发方式
14 - 读配置文件的步骤（先确认 mock profile / H2 配置）
15 - 测试清单规划的输出格式（表格含 IT 编号/场景/验证点/优先级）
16 - 标准的 mock 策略决策表
17 - 测试基类模板（@SpringBootTest + mock profile + MockMvc 标准配置）
```

```
18 - 场景递进原则（从最简单开始）
19 - 已知 bug 的处理方式（先写红测试，再修 bug）
```

固化后放在 `.claude/skills/integration-test.md`。下次写 Agent 模块的集成测试，直接：

```
复制代码
1 ./integration-test agent模块
```

流程自动复用，包括先读配置、分优先级规划、递进执行这些步骤。

总结

这一讲做完，你手上有四样东西：

1. **方法论**：质量保证是一个体系，不是 AI 能直接闭环的。
2. **CLAUDE.md 里的核心链路地图和单测规范**：所有模块共享的判断基础，后续所有测试都从这里出发。
3. **单测 SKILL**：任何 Service 方法一键触发，Claude Code 自动走完整流程，包括主动检查计划里的技术问题。
4. **集成测试 SKILL**：任何模块一键触发，流程自动复用。

这套体系解决的是前三层：**核心链路识别、单元测试、集成测试**。第四层混沌测试，故障注入验证系统在真实混乱条件下能不能撑住，这一讲没有展开，但它是存在的。当系统进入维护阶段、有了真实用户负载之后，混沌测试是质量体系的下一个自然延伸。

80–90% 的代码可以放心给 AI 写，前提是你知道剩下的 10–20% 在哪里，哪些地方要自己把关。

体系是工具，判断力才是核心。

思考题

1. 按照这一讲的判断框架，审视你现在工作中的一个项目：哪些代码适合给 AI 写，哪些需要自己把关？核心链路是哪几条？用 Step 0 的提示词让 Claude Code 帮你识别，再对比你自己的判断，看两者有没有差异。
2. 双模型 review 的提示词可以针对场景调整。如果要 review 的是安全敏感的代码，比如 Hify 的鉴权逻辑，review 维度应该怎么变？试着写一个专门针对安全 review 的提示词。
3. 混沌测试的场景设计依赖对系统风险的理解。基于 Step 0 生成的风险地图，Hify 最应该优先验证哪三个混沌场景？为什么？

期待你的分享！如果今天的课程让你有所收获，也欢迎转发给有需要的朋友，邀请他来一起学习，我们下节课再见！

© 版权归极客邦科技所有，未经许可不得传播售卖。 页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (3)

最新　精选

Geek_33d5cb
2026-05-03 来自湖南

老师，先分析代码再补充单元测试是不是和 TDD 违背了？

> 作者回复: 我个人习惯是没有特别严格的顺序。这点可能比较看个人的习惯。老实说，TDD的话，我个人也没有深入去学过。
> 我自己的话，先分析代码，再补充单元测试，更符合我自己编码过程的习惯。主要是因为很多时候代码不写出来，单元测试我也不知道要怎么写。

 　  2



许凯
2026-05-06 来自江苏

是不是不放在claude.md里面比较好，另开一个markdown文档

作者回复: 是的，独立到一个单独的md文档比较好。我一般放在docs下，新开对应的文档。然后在Claude.md中引用这个文档。让Claude Code 启动时去读这些独立的文件。

1



迪马

2026-05-31 来自浙江

不应该开发前就先分析核心链路和风险集中区域以及测试重心了吗，这样还可以提前指导开发

作者回复: 在05讲里面就做了这个事情，我们分析了“性能瓶颈预判”，这一步就是做了这个事情

共 2 条评论>